FRONTEND DEVELOPER ASSESSMENT

Hospital Feedback Analytics Dashboard

Time Limit: 24 Hours | Stack: React / Next.js | AI Tools: Allowed

Candidate must be able to explain all submitted code

1. Overview

This assessment requires you to design and build a Hospital Feedback Analytics Dashboard — a production-quality web application that helps hospital management monitor patient satisfaction, track doctor performance, manage complaints, and analyze service quality across departments.

The goal is not just to complete a checklist. We want to see how you think about architecture, component design, data flow, and user experience. The final product should be something you would be confident presenting to a real client.

2. What We Evaluate

Your submission will be reviewed across the following dimensions:

- Frontend development skills — component structure, code clarity, and reusability
- API integration — how you fetch, handle, and display data
- Data visualization — chart selection, accuracy, and presentation
- Dashboard architecture — folder structure, routing, and state management
- UI/UX — layout, spacing, responsiveness, and overall user experience
- Code quality — naming conventions, readability, and consistency

3. Tech Stack

Mandatory

- React.js or Next.js
- API integration (REST or mock data source)
- Responsive UI (mobile, tablet, desktop)
- Charts and data visualizations

Preferred (bonus consideration)

- TypeScript
- Tailwind CSS
- Zustand or Redux for state management
- TanStack Query for data fetching

4. Data Source

You may use any of the following for your data layer. Mock data is fully acceptable.

- Supabase
- Firebase

- MockAPI or JSON Server
- Static JSON files
- Your own backend (Node.js, Express, etc.)

Note: Whichever data source you choose, structure your data properly. Follow the schema defined in Section 6 for feedback records.

5. Application Pages

The dashboard should be organized into the following pages. Each page should have its own route.

Page 1	Dashboard — High-level overview with stat cards and summary charts
Page 2	Feedback Analytics — Detailed trend analysis and rating breakdowns
Page 3	Doctor Performance — Sortable, searchable, and filterable doctor rankings
Page 4	Complaints Management — Complaint tracking, categories, and resolution status
Page 5	Settings (Optional) — Theme preferences or user configuration

6. Data Schema

Each feedback record in your data source must follow this structure:

```
{
  "id":           101,
  "patientName":  "Amit Kumar",
  "department":   "Cardiology",
  "doctor":       "Dr. Sharma",
  "rating":       4,
  "feedback":     "Doctor was attentive and the treatment was effective.",
  "sentiment":    "Positive",
  "date":         "2026-05-10",
  "status":       "Resolved"
}
```

Generate a realistic mock dataset of at least 50–80 records covering multiple departments, doctors, ratings, and sentiments. This will make your charts meaningful.

7. Core Features

All seven features below are required. Build them in order of priority — the overview and feedback sections are most important.

7.1. Overview Statistics Cards

Cards to Display	Design Notes
• Total Feedbacks received • Average Rating (out of 5) • Positive Feedback percentage • Negative Feedback percentage • Total Complaints logged • Number of Active Departments	• Place cards at the top of the Dashboard page • Show percentage change vs. previous period if possible • Use icons to differentiate each card • Keep layout clean and scannable

7.2. Patient Feedback Analytics

Data to Visualize	Suggested Chart Types
• Daily and monthly feedback volume trend • Rating distribution (1 to 5 stars) • Positive vs. Negative split over time • Feedback by category or department	• Line Chart — feedback trend over time • Bar Chart — rating distribution • Pie or Donut Chart — sentiment split • Area Chart — volume over time

7.3. Department Performance

Departments to Include	Metrics per Department
• Emergency • Cardiology • Neurology • Orthopedics • Pediatrics • ICU	• Average rating • Total reviews received • Complaint count • Suggested: Horizontal bar chart or comparison table

7.4. Doctor Performance Dashboard

Fields to Display	Required Features
• Doctor name • Department • Consultation rating • Total patients seen • Complaints received • Overall feedback score	• Ranking by rating or feedback score • Search by doctor name • Sort by any column • Filter by department

7.5. Feedback Table

Table Columns	Required Features

• Patient Name • Department • Doctor • Rating • Feedback Message (truncated) • Sentiment • Date	• Pagination (10–20 rows per page) • Global search • Column sorting • Filter by department, sentiment, or rating

7.6. Complaint Analytics	
Metrics to Show	**Suggested Charts**
• Total complaints by category • Resolution status (Pending / In Progress / Resolved) • Average resolution time in days • Month-over-month complaint trend	• Pie or Donut Chart — complaints by category • Stacked Bar Chart — status breakdown over time

7.7. Sentiment Analysis	
Categories	**Implementation Options**
• Positive • Neutral • Negative	• Rule-based mock logic is acceptable • Optional: integrate an open-source sentiment API • Donut Chart for current distribution • Line or Area Chart for trend over time

8. Responsive Design

The dashboard must be fully functional and visually clean on all screen sizes. Test your layout on at least these breakpoints:

- Desktop — 1280px and above
- Tablet — 768px to 1279px
- Mobile — below 768px

On smaller screens, charts should resize gracefully, tables should scroll horizontally, and navigation should collapse into a mobile menu.

9. UI Expectations

We are not looking for a flashy design. We are looking for a clean, well-organized interface that is easy to navigate. Specifically:

- Consistent spacing, typography, and alignment throughout
- Proper loading states — use skeleton loaders or spinners while data loads
- Meaningful empty states when no data is available for a section
- Reusable components — avoid writing the same UI logic in multiple places
- Clear navigation between pages
- No broken layouts on any screen size

10. Suggested Libraries

You are free to use any library, but these are well-suited for this project:

UI Components	Shadcn UI, Material UI, or Ant Design
Charts	Recharts, ApexCharts, or Chart.js
Tables	TanStack Table or AG Grid
State	Zustand or Redux Toolkit
Data Fetching	TanStack Query (React Query) or SWR
Styling	Tailwind CSS or CSS Modules

11. Bonus Features

The following are optional. Implementing any of them will earn additional marks, but only if the core features are solid first.

- Dark mode toggle
- Authentication with role-based access (Admin, Manager, Doctor)
- Export data as CSV or PDF
- Real-time feedback updates using WebSocket or polling
- Notification system for new complaints or critically low ratings
- AI-generated summary or insight for each section

12. Evaluation Criteria

Criteria	Weightage
UI / UX Design	**20%**
Code Quality	**20%**
Responsiveness	**15%**

API Integration	**15%**
Charts & Visualization	**15%**
Architecture	**10%**
Performance	**5%**

13. Submission Requirements

Submit all of the following before the deadline:

1. GitHub Repository — public or shared with the reviewer. Commit history should be clean and reflect your development process.
2. Live Deployment Link — deploy on Vercel, Netlify, Railway, or any platform of your choice.
3. README.md — include a project description, list of features implemented, tech stack used, and any known limitations.
4. Local Setup Instructions — clear steps so the reviewer can run the project locally without guesswork.

14. Important Notes

- AI tools are permitted. However, you will be expected to explain your code during the review. If you cannot explain it, it will count against you.
- Use a proper folder structure. Suggested: components/, pages/ or app/, hooks/, store/, utils/, types/.
- Do not hardcode data directly into UI components. Drive your rendering from a data layer.
- Write code as if another developer will maintain it. Clarity matters more than cleverness.
- Focus on completing the core features well before attempting bonus features.
- Test your application before submitting. Broken features will be penalized.

Expected Outcome: The final submission should function and feel like a production-level analytics dashboard that a hospital management team could use to monitor patient satisfaction and service quality on a daily basis.